

Aprendizaje no supervisado

Jorge Gallego

Inter-American Development Bank

Bogotá Summer School in Economics — 2026

Agenda de la sesión

- Hasta ahora, aprendizaje **supervisado**: predecir un *outcome* conocido
- Hoy cambiamos de mundo: **no** hay etiquetas, queremos **descubrir estructura**
 1. Introducción al aprendizaje no supervisado
 2. Clustering (k-medias)
 3. Detección de anomalías
 4. Representación de datos y *embeddings*

Parte 1

¿Qué es el aprendizaje no supervisado?

Aprendizaje no supervisado

- En el supervisado teníamos un *outcome* y que queríamos predecir
- En el **no supervisado** no hay *y*: solo tenemos las características
- El objetivo no es predecir, sino **describir** y **descubrir patrones** en los datos
- Ninguna variable es más importante que las demás: las miramos todas en conjunto

Tres tareas que veremos

1. **Clustering**: agrupar objetos similares (segmentar)
2. **Detección de anomalías**: encontrar los casos “raros”
3. **Representación / reducción de dimensión**: resumir muchas variables en pocas, más útiles
 - Como no hay etiquetas, evaluar el resultado es más difícil: mucho depende de la **interpretación** del analista

¿Para qué sirve?

- **Segmentación** de clientes, votantes o mercados
- **Detección de fraude**, fallas o intrusiones (lo raro)
- **Visualización** y **compresión** de datos de alta dimensión
- **Preprocesamiento**: crear mejores variables para un modelo supervisado posterior

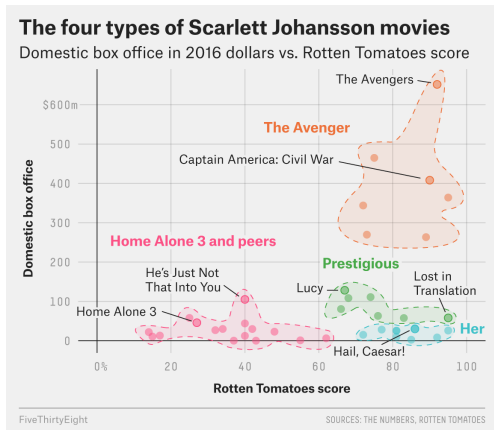
Parte 2

Clustering con k-medias

Clustering

- A veces conviene **agrupar** objetos en categorías que no conocíamos de antemano
- En marketing: segmentar mercados para una estrategia por grupo
- En campañas políticas: identificar tipos de votantes
- El clustering divide los datos en **clusters**: grupos de objetos similares entre sí y distintos de los demás

Un ejemplo cotidiano



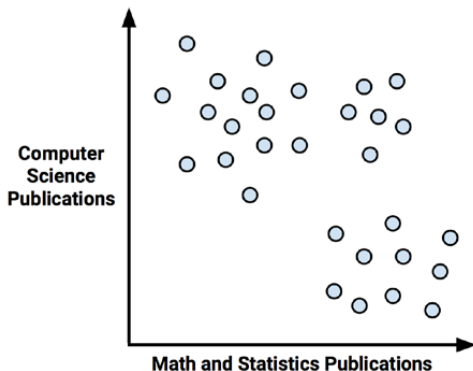
Las películas de un actor podrían agruparse por género sin decirle al algoritmo cuáles son los géneros.

Clustering: clasificación no supervisada

- Es **no supervisado**: no decimos ex ante cómo deben verse los grupos
- No sirve para **predecir**, sino para **descubrir** estructura
- Los grupos que devuelve el computador **no tienen significado explícito**: la interpretación la pone el analista
- ¿Cómo decide el computador dónde están las fronteras de los grupos?

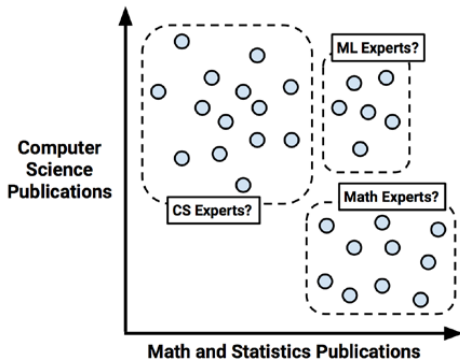
Ejemplo: conferencia de *data science*

- Queremos sentar a los asistentes según su especialidad, pero no sabemos a qué área pertenecen
- Los describimos por dos características: cuánto publican en **computer science** y en **matemáticas/estadística**



Ejemplo: conferencia de *data science*

¿Se ven grupos? Computer scientists, matemáticos/estadísticos, y *machine learning* (mucho de ambos).

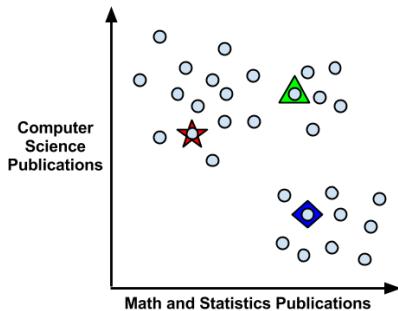


El algoritmo de k-medias

- El más usado para clustering; emparentado con kNN
- Asigna cada uno de los n ejemplos a uno de k clusters
- El número k se fija **ex ante** (de forma heurística)
- Busca **minimizar** las diferencias dentro de cada cluster y **maximizar** las diferencias entre clusters

k-medias: elegir centros

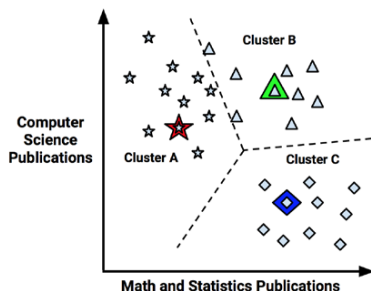
- Se eligen al azar k puntos como centros iniciales (aquí $k = 3$):



k-medias: asignar al centro más cercano

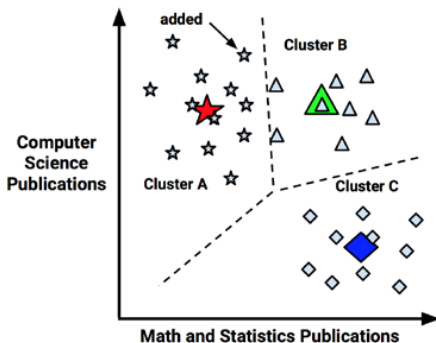
- Cada punto se asigna al centro más cercano (distancia euclídea): se forma el **diagrama de Voronoi**

$$\text{dist}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



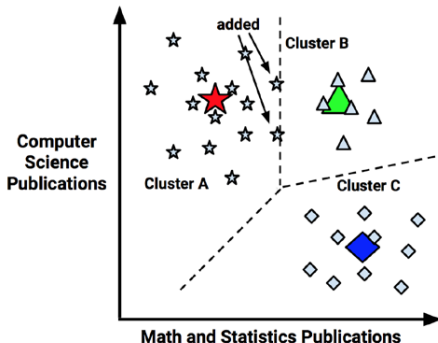
k-medias: actualizar los centroides

- Cada centro se mueve al **centroide** (la posición promedio) de los puntos de su cluster:



k-medias: iterar hasta converger

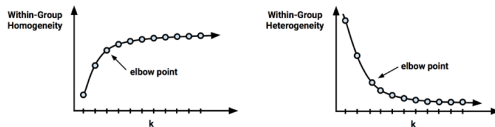
- Con los nuevos centros se reasignan los puntos; algunos cambian de cluster
- Se repite hasta que ya no haya cambios:



¿Cuántos clusters?

- k es una decisión crucial (y el algoritmo también es sensible a los centros iniciales)
- Si hay un **prior**, úsalo (categorías de negocio, número de mesas, de campañas...)
- Regla de dedo: $k = \sqrt{n/2}$
- **Método del codo**: aumentar k hasta que la ganancia marginal en homogeneidad se aplane

Método del codo



Nos quedamos en el “codo”: donde añadir clusters ya casi no reduce la heterogeneidad.

Al taller

- Segmentaremos a **30.000 adolescentes** de una red social según las palabras de su perfil
- Usaremos $k = 5$ (inspirados en *The Breakfast Club*: princesas, atletas, cerebros, etc.)
- Veremos cómo el algoritmo recupera grupos interpretables sin que le digamos nada de antemano
- Base: `teens.csv`, con la función `kmeans()`

Parte 3

Detección de anomalías

¿Qué es una anomalía?

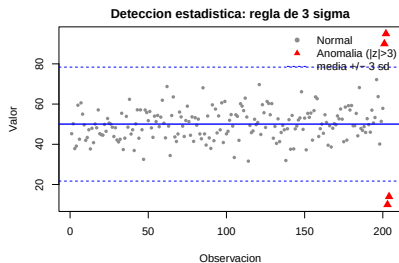
- Una **anomalía** (*outlier*) es un caso que se aparta mucho del patrón general
- A menudo lo raro es lo más interesante:
 - ▶ transacciones **fraudulentas** con tarjeta
 - ▶ **fallas** de una máquina o un sensor
 - ▶ **intrusiones** en una red; errores en los datos
- Suele ser un problema **no supervisado**: no tenemos ejemplos etiquetados de “anomalía”

Cuatro familias de métodos

- **Estadístico:** lo que cae lejos de lo típico (z-score, rango intercuartílico)
- **Por distancia:** puntos lejos de sus vecinos o del centro de su cluster
- **Por densidad:** puntos en zonas poco pobladas (LOF)
- **Por modelo:** aprender la “forma normal” y marcar lo que no encaja (*isolation forest, one-class SVM*)

Enfoque estadístico

- Univariado: marcar valores con $|z| > 3$ (regla de 3 sigma) o fuera del rango intercuartílico (Tukey)
- Multivariado: la **distancia de Mahalanobis** mide qué tan “raro” es un punto teniendo en cuenta la correlación entre variables
- Simple y rápido, pero asume una forma (p. ej. normal)



Enfoque por distancia

- Idea: un punto es anómalo si está **lejos** de los demás
- Distancia al k -ésimo vecino más cercano: a mayor distancia, más aislado
- O distancia al **centroide** de su cluster (conecta con k-medias): los puntos lejos de todos los centros son sospechosos
- Limitación: falla cuando hay zonas con densidades muy distintas

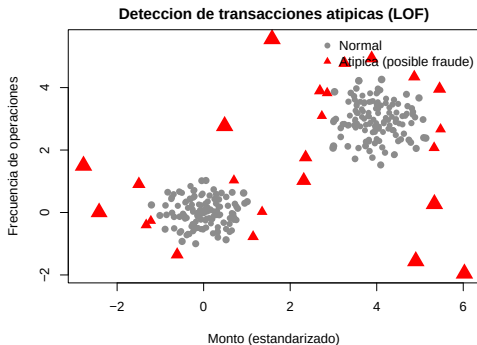
LOF: Local Outlier Factor

- Idea: comparar la **densidad local** de un punto con la de sus vecinos
- Si un punto está en una zona **mucho menos densa** que sus vecinos, es sospechoso
- Devuelve un **puntaje**:
 - ▶ $LOF \approx 1$: densidad parecida a la de sus vecinos (normal)
 - ▶ $LOF \gg 1$: mucho más aislado que sus vecinos (anomalía)
- Ventaja sobre el enfoque por distancia: detecta anomalías **locales**, no solo las globales

LOF: ¿cómo se calcula? (intuición)

1. Para cada punto, se miran sus k vecinos más cercanos
 2. Se estima su **densidad local**: qué tan apiñados están esos vecinos a su alrededor
 3. Se compara esa densidad con la densidad **de** sus vecinos
 4. El **LOF** es la razón entre ambas: si la del punto es mucho menor, el cociente es alto
- Por eso un punto solitario al borde de un grupo denso tiene LOF alto, aunque no esté “tan lejos” en distancia absoluta

LOF en acción



Los puntos con LOF alto (rojos) están en zonas poco densas, lejos de los grupos.

Isolation Forest

- Un enfoque **por modelo**, basado en árboles (conecta con random forests)
- Idea: aislar cada punto haciendo **cortes aleatorios** en las variables
- Un punto **anómalo**, al estar apartado, se aísla con **muy pocos** cortes
- Uno **normal**, en una zona densa, requiere muchos cortes
- El número promedio de cortes mide la anomalía. Es rápido y escala bien a muchos datos

¿Cómo evaluar sin etiquetas?

- Sin etiquetas de “anomalía”, validar es difícil: a menudo se **inspeccionan a mano** los casos con mayor puntaje
- Si tenemos **algunas** etiquetas (p. ej. fraudes confirmados), es un problema **semi-supervisado**: podemos medir cuántos de los *top-N* marcados eran reales (precisión@N)
- No hay umbral universal: cuántos casos marcar depende del **costo** de cada error
- Como se basa en distancias, conviene **escalar** las variables (como en kNN)

Aplicaciones y al taller

- **Fraude** financiero y de tarjetas; **ciberseguridad** (intrusiones)
- **Mantenimiento predictivo** (sensores que anticipan fallas)
- **Salud** (lecturas anómalas) y **calidad de datos** (errores de captura)
- **Al taller**: aplicaremos LOF (paquete dbscan) a un conjunto de **transacciones** (en 2D) para señalar operaciones atípicas (posible fraude)

Parte 4

Representación de datos y *embeddings*

El reto de la alta dimensión

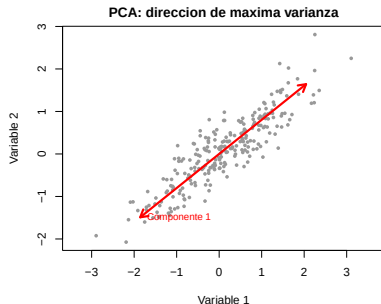
- Muchos problemas tienen **decenas o miles** de variables
- Difíciles de **visualizar**, costosas de procesar, y con la **maldición de la dimensionalidad**
- Pero muchas variables están **correlacionadas**: hay redundancia
- ¿Podemos **resumir** la información en muchas menos dimensiones, perdiendo poco?

Aprendizaje de representación

- En vez de usar las variables crudas, buscamos una **representación** más compacta y útil de los datos
- Es la versión no supervisada del *feature extraction* que vimos en desempeño
- La técnica clásica y más intuitiva: **Análisis de Componentes Principales** (PCA)
- (Y la base conceptual de los *embeddings* del aprendizaje profundo)

PCA: la idea

- PCA encuentra nuevas variables (**componentes**) que son combinaciones lineales de las originales
- La 1.^a componente apunta en la dirección de **máxima varianza**
- Las siguientes son ortogonales y captan la varianza restante

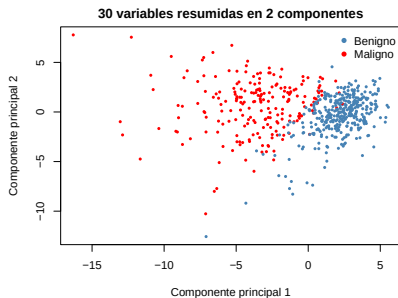


PCA: ¿cómo funciona?

- Buscamos direcciones que maximicen la **varianza** de los datos proyectados
- Esas direcciones son los **autovectores** de la matriz de covarianzas; cada **autovalor** dice cuánta varianza captura su dirección
- Las componentes son **ortogonales** (no correlacionadas) y se ordenan de mayor a menor varianza
- Intuición, sin fórmulas: “girar los ejes” para alinearlos con la mayor dispersión de la nube de puntos

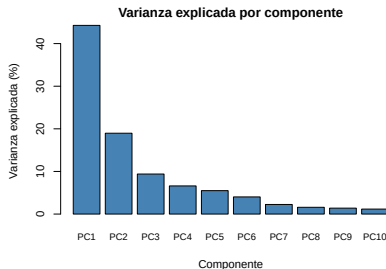
PCA: de 30 variables a 2

- Tomamos las 30 variables del cáncer de seno (sesión 1)
- Las proyectamos en sus 2 primeras componentes
- Aunque PCA **no** usa el diagnóstico, los benignos y malignos quedan casi separados



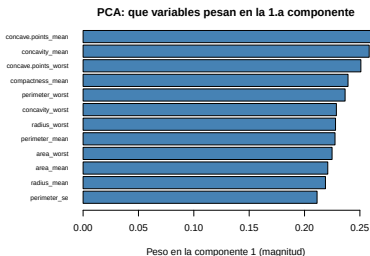
¿Cuántas componentes?

- Cada componente explica una fracción de la **varianza** total
- Aquí, las 2 primeras ya capturan $\sim 63\%$
- Nos quedamos con las que acumulan “suficiente” (otra vez, el codo)



Interpretar las componentes

- Cada componente es una **mezcla** de las variables originales
- Los **pesos** (*loadings*) dicen cuánto aporta cada variable
- Mirándolos le damos sentido a la componente (aquí, PC1 $\approx$ “tamaño/severidad” del tumor)



¿Para qué usar PCA?

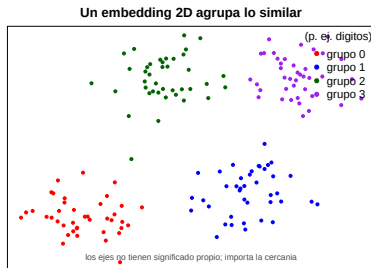
- **Compresión:** menos columnas, menos cómputo y almacenamiento
- **Visualización:** graficar en 2 o 3 componentes
- **Quitar multicolinealidad** antes de una regresión (las componentes no están correlacionadas)
- **Preprocesamiento** para otros modelos
- Costo: las componentes son **menos interpretables** que las variables originales

Más allá de PCA: t-SNE y UMAP

- PCA es **lineal**. Para estructuras curvas hay métodos no lineales
- **t-SNE** y **UMAP**: pensados sobre todo para **visualizar** datos de alta dimensión en 2D
- Preservan la **vecindad local**: puntos parecidos quedan cerca
- Muy usados para explorar imágenes, texto o datos genéticos

t-SNE/UMAP: cómo leerlos

- Lo **cercano** es similar; los grupos saltan a la vista
- **Cuidado**: NO conservan distancias globales ni tamaños de cluster
- No interpretes el tamaño de los grupos ni qué tan separados están como algo literal



Embeddings

- Un **embedding** representa un objeto complejo como un **vector denso** de pocos números
- **word2vec**: palabras como vectores; las de significado parecido quedan cerca
- Y las relaciones se vuelven **aritmética**: “ $\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$ ”

Embeddings: relaciones con significado



Autoencoders

- Una red neuronal que aprende a **comprimir** los datos y luego **reconstruirlos**
- La capa central (un “cuello de botella”) es una **representación compacta** aprendida
- Es la versión **profunda y no lineal** de la reducción de dimensión
- Las redes neuronales aprenden embeddings de palabras, imágenes, usuarios, productos...
- Es el puente hacia la última sesión: **aprendizaje profundo**

Al taller

- Aplicaremos **PCA** a las 30 variables del cáncer de seno (`wisc_bc_data.csv`)
- Con la función `prcomp()` de R base
- Veremos la varianza explicada y proyectaremos los datos en 2 dimensiones
- Comprobando cuánta estructura se conserva al reducir de 30 a 2

Resumen de la sesión

- El aprendizaje **no supervisado** descubre estructura sin etiquetas
- **Clustering** (k-medias): agrupa objetos similares; elegir k es clave
- **Detección de anomalías**: estadística, distancia, densidad (LOF) o modelos (isolation forest); el contexto define el umbral
- **Representación**: PCA resume muchas variables en pocas componentes; t-SNE/UMAP visualizan; los *embeddings* y *autoencoders* extienden la idea hacia el aprendizaje profundo